

REQUIREMENTS DOCUMENT

Autonomous Pentesting Agent

LangGraph / LangChain Implementation

Document Version	1.0
Status	Draft
Date	22 May 2026
Project	Autonomous Pentesting Agent
Stack	Python 3.11+ · LangGraph · LangChain · Docker

1. Project Overview

This document defines the functional and non-functional requirements for an autonomous AI agent that performs a complete web application authentication lifecycle. The agent is implemented using LangGraph/LangChain and targets a FastAPI application running in Docker. The primary goal is to demonstrate the agent's ability to act autonomously across multi-step tasks, recover from errors gracefully, detect anomalous server behaviour, and produce a structured audit report of all actions taken.

2. Objectives

1. Authenticate to the target application using provided credentials.
 2. Change the user's password to a newly generated or supplied value.
 3. Log out of the session, invalidating the current token.
 4. Re-authenticate with the new password to validate the change succeeded.
 5. Produce a structured report (JSON) summarising success/failure at each step, errors encountered, and decisions made.
 6. (Optional) Detect anomalous server behaviour such as not requiring the current password during a change-password operation.
-

3. Target Application

A FastAPI application is provided as a Docker image. The agent must interact exclusively with its HTTP API. The following endpoints constitute the full contract:

Method	Endpoint	Description	Auth Required
POST	/login	Accepts username + password; returns session token.	No
POST	/change-password	Accepts current + new password; requires valid token.	Yes (Bearer)
POST	/logout	Invalidates the current session token.	Yes (Bearer)
GET	/me	Returns authenticated user info. Used for validation.	Yes (Bearer)

The application is started locally via docker compose up as described in its README.

4. Functional Requirements

4.1 Agent Workflow

The agent must execute the following steps in sequence, with each step's outcome influencing subsequent decisions:

Step	Name	Description	Validation
1	Login	POST /login with username and password. Store returned session token.	Token received; /me returns correct user.
2	Change Password	POST /change-password with current password and new password. Token must be included in headers.	HTTP 200; no error response body.
3	Logout	POST /logout with current token. Token must be invalidated.	Subsequent /me call returns 401.
4	Re-authenticate	POST /login with username and new password.	Token received; /me returns correct user.
5	Report	Emit structured JSON report of all steps.	Report file written to output path.

4.2 Error Handling

The agent must handle all foreseeable failure modes and decide autonomously whether to retry, abort, or report:

- Invalid credentials (401) — report failure and abort; do not retry with same credentials.
- Endpoint unavailable / network timeout — retry up to a configurable number of times (default: 3) with exponential back-off before aborting.
- Token expired mid-flow — attempt to re-authenticate and resume from the failed step.
- Unexpected HTTP status codes — log the error, record the decision taken, and abort gracefully.
- Change-password endpoint does not require current password — flag as anomalous behaviour (see §4.4) and record in report.

4.3 Structured Report Output

Upon completion (success or failure), the agent must emit a JSON report to stdout and optionally to a file. The report must include:

- Overall status: success | partial_failure | failure.
- Per-step result: step name, status, HTTP status code, error message (if any), and timestamp.
- Decisions taken by the agent at each decision point (e.g. 'retried after 502', 'aborted after 3 failed login attempts').

- List of anomalies detected (may be empty).
- Total elapsed time in milliseconds.

Example report structure (JSON):

```
{ "status": "success", "steps": [ { "name": "login", "status": "ok", ... } ],
  "anomalies": [], "elapsed_ms": 1240 }
```

4.4 Anomaly Detection (Optional)

The agent should optionally detect and report the following anomalous server behaviours:

- Change-password endpoint accepts a request without a valid current password.
- Logout endpoint returns success but the token remains valid (detectable via a follow-up /me call).
- The session token does not change after re-authentication.

Each detected anomaly must be recorded in the report's anomalies array with a description and the evidence observed.

5. Non-Functional Requirements

ID	Category	Requirement	Priority
NFR-01	Language	Python 3.11 or higher.	Mandatory
NFR-02	Framework	LangChain and LangGraph must be used for agent orchestration.	Mandatory
NFR-03	Containerisation	Project must run via docker compose up or docker run without additional setup.	Mandatory
NFR-04	Documentation	README.md with clear setup and execution instructions.	Mandatory
NFR-05	Structured Logging	All agent decisions and tool calls must be logged in a structured format (JSON lines recommended).	Mandatory
NFR-06	Configurability	Base URL and credentials must be injectable via environment variables or CLI arguments.	Mandatory
NFR-07	Idempotency	The agent must be safely re-runnable (e.g., handle the case where the password was already changed).	Recommended
NFR-08	Testability	At least one test (unit or integration) demonstrating agent behaviour on a known scenario.	Recommended
NFR-09	Execution Time	Complete flow must finish within 60 seconds under normal network conditions.	Recommended
NFR-	Dependency	Dependencies declared in requirements.txt or	Mandatory

ID	Category	Requirement	Priority
10	Management	pyproject.toml. Pinned versions preferred.	

6. Technical Architecture

6.1 Agent Graph (LangGraph)

The agent must be modelled as a directed state graph in LangGraph. Recommended node breakdown:

- login_node — calls POST /login; stores token in graph state.
- validate_login_node — calls GET /me to confirm authentication.
- change_password_node — calls POST /change-password with current + new password.
- logout_node — calls POST /logout; clears token from state.
- reauth_node — calls POST /login with new password.
- validate_reauth_node — calls GET /me to confirm new credentials work.
- report_node — assembles and emits the structured JSON report.
- error_handler_node — intercepts failures; decides retry / abort and transitions accordingly.

6.2 State Schema

The LangGraph state must carry at minimum:

- base_url, username, current_password, new_password — input configuration.
- session_token — updated at each authentication step.
- step_results — list of per-step result objects accumulated throughout the run.
- anomalies — list of detected anomalies.
- error — last error message, if any.
- retry_count — per-node retry counter.

6.3 Tools / HTTP Client

Each endpoint interaction should be wrapped as a LangChain tool (or equivalent callable) to allow the LLM or graph logic to invoke them in a structured, observable way. The HTTP client (e.g. httpx or requests) must enforce timeouts on all calls.

7. Deliverables

Artefact	Description	Required
agent/	LangGraph agent source code.	Yes
Dockerfile	Builds the agent image.	Yes

Artefact	Description	Required
docker-compose.yml	Orchestrates agent + target app.	Yes
README.md	Setup, configuration, and execution instructions.	Yes
requirements.txt / pyproject.toml	Pinned Python dependencies.	Yes
tests/	Unit or integration tests.	Recommended
sample_report.json	Example output report.	Recommended

8. Acceptance Criteria

The implementation is considered complete when all of the following are true:

- Running docker compose up starts both the target app and the agent without manual intervention.
- The agent completes the full 4-step flow (login → change password → logout → re-login) against the provided FastAPI app.
- A valid JSON report is produced, capturing the result of every step and all decisions taken.
- The agent handles at least: invalid credentials, endpoint timeout, and token expiry — without crashing.
- Structured logs are emitted to stdout during execution.
- The README is sufficient for a new developer to run the project from scratch.

9. Constraints & Assumptions

- The target FastAPI application is treated as a black box; the agent must not read its source code at runtime.
- A valid username and initial password will be supplied via environment variables (AGENT_USERNAME, AGENT_PASSWORD).
- The new password value may be hardcoded, randomly generated, or supplied as a third environment variable (AGENT_NEW_PASSWORD).
- The agent runs in a single-pass mode (no interactive UI); execution is triggered by docker compose up or a CLI entrypoint.
- Estimated implementation effort is 3–4 hours; scope should be sized accordingly. Simplicity and correctness are prioritised over feature completeness.

10. Out of Scope

- Multi-user or concurrent session testing.
- SQL injection, XSS, or any active exploit payloads.
- A graphical or web-based reporting UI.

- Integration with external SIEM or ticketing systems.
- Persistent storage of results across runs.